

Contents

1 Chapter 13 — Behavioral Analysis Pipelines on Real Hand Histories: Experiment Report	1
1.1 Part I — THE DATA AND THE PIPELINE	2
1.2 Experiment 1 — Parser and validators	2
1.3 Experiment 2 — Player statistics, their minimum samples, and the gap to a reference	4
1.4 Part II — MODELING BEHAVIOR	6
1.5 Experiment 3 — Behavioral cloning	6
1.6 Experiment 4 — A player2vec-style embedding and re-identification	9
1.7 Experiment 5 — Style clusters, temporal stability, online typing	10
1.8 Part III — FAIR PLAY	12
1.9 Experiment 6 — Collusion injected into real sessions	12
1.10 Experiment 7 — Can behavior alone pick out Pluribus?	14
1.11 Validation summary	15
1.12 Prediction $\leftrightarrow$ reality reconciliation	16
1.13 Trustworthiness and sample adequacy	17
1.14 Limitations (ranked by how much they affect the conclusions)	17
1.15 Conclusions and research directions	17
1.16 Reproduction	18

1 Chapter 13 — Behavioral Analysis Pipelines on Real Hand Histories: Experiment Report

Testbed: real online poker. The plan assumes Playtech hand histories, which the candidate does not have yet. The substitute is public: **2,032,655 no-limit hold'em hands played on the iPoker Network at \$0.50/\$1 blinds in July 2009**, with obfuscated player IDs, from the MIT-licensed PHH dataset; and the **10,000 released hands of Pluribus** against professionals, the one sample in which a bot is known. iPoker was Playtech's own network in 2009 (launched by Playtech in 2004, described in its 2009 annual report as "Playtech's exclusive poker network"), so this is the closest public sample of a Playtech network. It is 2009 data, not Playtech's current data; the Playtech dataset remains the validation this chapter prepares for. On this data the chapter builds a parser with two validators, player statistics with measured minimum samples, a behavioral cloning baseline, a player2vec-style embedding, style clusters with online Bayesian typing, a collusion detector tested on colluders injected into real sessions, and a bot-detection test on Pluribus.

PhD connection: this is the practical core of Contribution #1 — representing, classifying, predicting and tracking behavior from raw logs — and the fair-play angle of the individual plan (collusion and bots). Collusion detection in the literature is scored offline, as a detection task, and is not coupled to the detecting agent's play; this chapter stays on that side of the line and measures what a detector can and cannot see.

Scope of results: every number below was measured on a real run and is read from

implementation/step13/implementation/results/ — build_IPN100.json, build_PLURIBUS.json, validator.json, player_stats.json, bc.json, bc_PLURIBUS.json, player2vec.json, clustering.json, collusion.json, collusion_mw.json, botdetect.json, derived.json — and from the run log implementation/step13/EXECUTION_NOTES.md. Training and sampling use seeds 0, 1 and 2; differences are reported as mean $\pm$ standard error over the three.

How to read this report. Part I builds and checks the data pipeline. Part II models behavior: cloning, embedding, clustering and online typing. Part III is fair play: collusion and bots. Each experiment runs *what we test* $\rightarrow$ *how* $\rightarrow$ *results* $\rightarrow$ *conclusion*. Contradicted predictions are kept and reconciled in *Prediction* $\leftrightarrow$ *reality reconciliation*.

1.1 Part I — THE DATA AND THE PIPELINE

1.2 Experiment 1 — Parser and validators

What we test. Can the hands be replayed exactly, and do the validators catch corrupted hands? (Raw step, Day 1; validation “parse 100 % of well-formed hands; validator catches injected errors”.)

How. `phh_parser.py` replays each hand with its own no-limit engine (turn order, minimum raise, stacks, board counts, side pots, payoffs). A second validator is pokerkit’s rules engine. For the injection test, 2,000 hands from 40 random iPoker files and 2,000 Pluribus hands were sampled; in every hand both validators accept, one error of each type was injected and the rejection rate counted (`validator.py`, seed 0).

Three data conventions had to be learned from failures before the replay was right: pokerkit reverses the blinds in heads-up hands; a negative blind is a *post* by a newly seated player (the first version counted it as a refund); and the iPoker records have no stacks (`inf`), so an all-in shows only as a street dealt with no betting on it. The parser accepts exactly that pattern (86,362 hands, 4.3 %).

Table 1: Parsing the two datasets.

	iPoker 100NL	Pluribus
hands in the files	2,032,655	10,000
parsed	2,027,882 (99.77 %)	10,000
rejected: truncated (4,130 + 619), out of turn (24)	4,773	0
players	21,453	14
decisions	15,576,700	91,356
showdowns with known / unknown cards	126,343 / 158,529	1,673 / 0

Table 2: Share of injected errors each validator rejects (868–1,997 hands per type).

Injected error	own	pokerkit	pokerkit, warnings as errors
duplicate card (iPoker / Pluribus)	100 % / 100 %	0 % / 3.8 %	52.9 % / 93.0 %
out of turn, below minimum raise, action after fold or after the end, unknown player	100 %	100 %	100 %
bet larger than the stack (Pluribus)	100 %	100 %	100 %
wrong board count (iPoker / Pluribus)	100 %	100 % / 99.0 %	100 % / 99.0 %
truncated hand	100 %	0 %	0 %
finishing stacks altered (Pluribus)	100 %	100 %	100 %

Results. The parser accepts 99.77 % of iPoker hands; every rejected hand is genuinely incomplete (for example, the small blind raises and the big blind never acts). pokerkit accepts those, because an unfinished hand is not an illegal one, and it only *warns* on a duplicate card (with warnings promoted to errors it still misses duplicates inside a single board deal). As a cross-check of the accounting, the parser’s net results equal pokerkit’s payoffs on 1,884 of 1,914 comparable iPoker hands; all 30 differences are showdowns where the cards were dealt face up but the show line says `sm ???? —` pokerkit then splits the pot, the parser evaluates the known cards. On Pluribus, 1,997 of 1,997 agree. pokerkit rejects 8 of 10,000 Pluribus hands: all are split pots in which the log awards half chips and pokerkit awards the odd chip.

Conclusion. The two validators check different things: pokerkit checks that every action is legal; the own validator also checks that the hand is complete and consistent. Using both, and comparing their payoffs, is what exposed the three conventions above. The remaining data limits are properties of the source, not of the parser: no stacks, no recorded winnings, and cards known in only 14 % of hands.

1.3 Experiment 2 — Player statistics, their minimum samples, and the gap to a reference

What we test. (a) How many hands each statistic needs before it describes the player rather than the cards (raw step: “need 500+ hands”). (b) The VPIP $\times$ PFR map. (c) How far real players are from a strong six-player strategy.

How. Every statistic is a count over its own opportunities (3-bet % over the times a player faced exactly one raise, c-bet % over the flops where the pre-flop raiser could bet first, and so on). Reliability is the split-half correlation across players: two disjoint random samples of m hands from each player with at least $2m$, 3 seeds (`player_stats.py`). For the gap there is no computed equilibrium of six-player no-limit hold’em to compare with, so the reference is **Pluribus’s own six-handed frequencies** over its 10,000 hands. Pluribus is not an equilibrium; it is the strongest six-player strategy with public hands. The gap is a frequency difference, not an expected-value figure.

Results. 2,906 regulars have at least 500 hands; they hold 84.6 % of all seats. The statistics need very different samples ($SE \leq 0.009$ over seeds):

Table 3: Split-half correlation of each statistic across players.

hands per half	50	100	200	500	1,000
VPIP	0.85	0.92	0.95	0.97	0.98
PFR	0.77	0.87	0.92	0.96	0.98
3-bet	0.38	0.54	0.67	0.82	0.88
c-bet	0.18	0.24	0.36	0.57	0.70
went to showdown	0.13	0.20	0.35	0.57	0.74
fold to c-bet	0.06	0.11	0.13	0.18	0.25

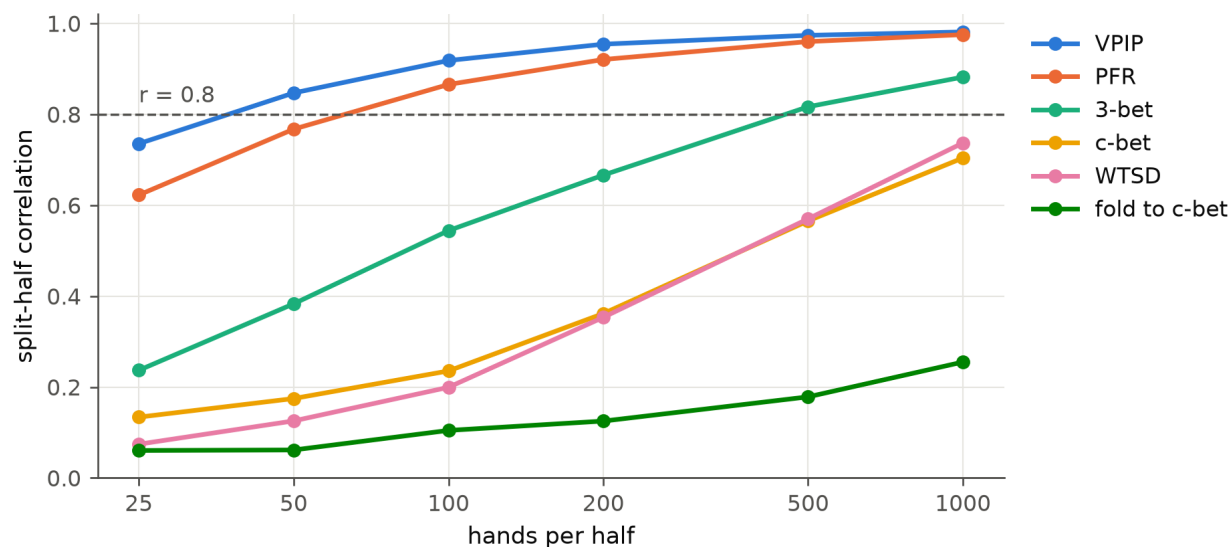


Figure 1: Split-half correlation of six statistics across the iPoker regulars as a function of the number of hands in each half (3 seeds; standard errors are smaller than the markers). VPIP and PFR pass $r = 0.8$ below 100 hands, 3-bet near 500; the post-flop statistics stay below 0.8 at 1,000 hands.

On the VPIP $\times$ PFR map (loose at VPIP ≥ 27.5 %, the midpoint of the plan's 20 % / 35 %; aggressive if PFR $\geq$ VPIP/2), the regulars split into 1,323 tight-aggressive, 730 loose-passive, 462 tight-passive and 391 loose-aggressive players. Win-at-showdown cannot be measured properly: the cards are known for a median 41 % of a regular's showdowns.

For the gap, 1,046 regulars have at least 500 six-handed hands. Their median mean-absolute difference from Pluribus over 11 pre-flop and flop frequencies is 0.093 (10th–90th percentile 0.062–0.141); the 13 professionals' median is 0.039, and only 0.19 % of the regulars are closer to Pluribus than the median professional. The direction is consistent: the 2009 regulars defend the big blind far less (VPIP in the big blind 22 points lower), open less from every position (3 to 9 points), 3-bet less, and c-bet more (+10.5 points) and fold to c-bets more (+7.1).

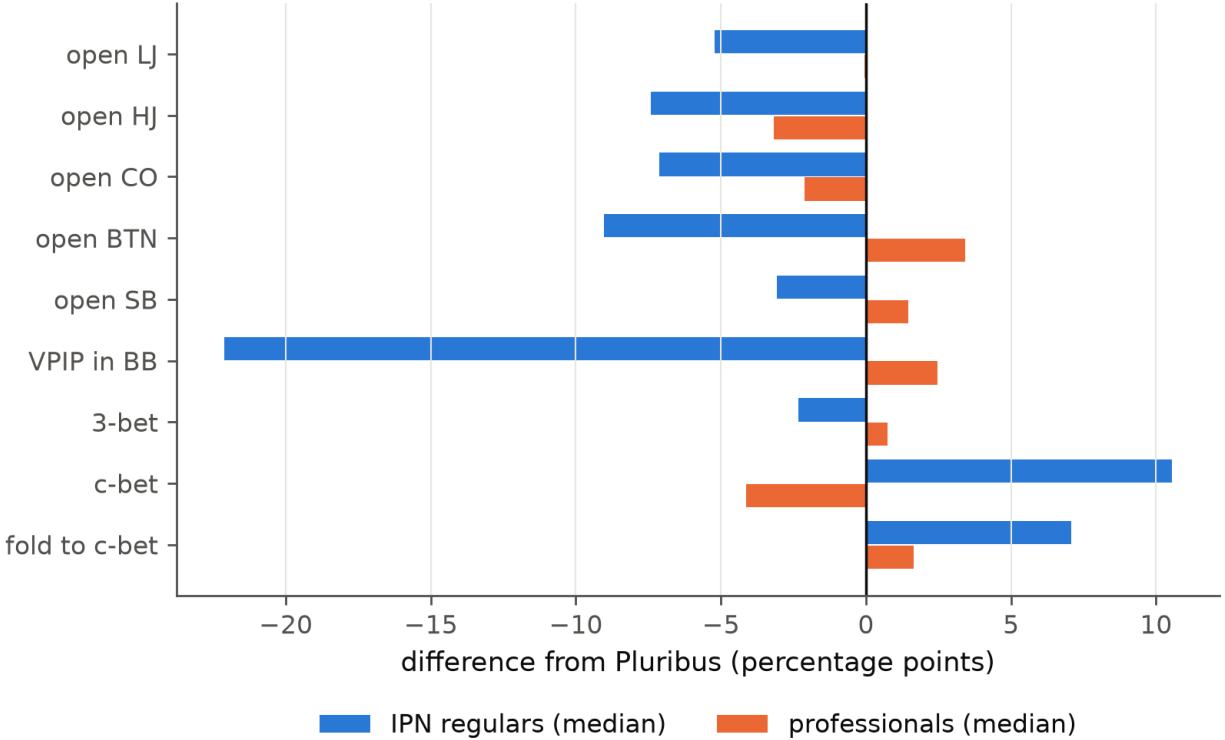


Figure 2: Median difference from Pluribus’s six-handed frequencies, for the 1,046 iPoker regulars with at least 500 six-handed hands and for the 13 professionals of the Pluribus games (percentage points). The regulars defend the big blind far less and c-bet far more; the professionals stay within a few points.

Conclusion. The plan’s single “500 hands” rule is right for VPIP, PFR and 3-bet and wrong for the post-flop statistics, which remain noisy after 1,000 hands; per-statistic minimum opportunities are used from here on (`config.MIN_OPP`). The gap separates the two populations cleanly and points to where an exploiting agent would look (big-blind defense, c-betting). It does not say how much such an agent would win: that needs an EV model of the game, which a frequency comparison cannot supply.

1.4 Part II — MODELING BEHAVIOR

1.5 Experiment 3 — Behavioral cloning

What we test. How well is a real decision predicted from the public state, and does knowing *who* acts help? (Raw step, Day 2: “accuracy > 55 %; by street, by position, by archetype, TAG > LAG > Fish”.)

How. Six action classes: fold, check, call, and a bet or raise that is small, medium or large relative to the pot. 42 public features: street, position, table size, players left and still to act, pot, price, raises so far, aggressor flags, board texture. Hole cards are not used on iPoker (no card is known in 86 % of hands, and known cards are selected). Legal-action masking for every model. Split by time: days 1–18 train (6 M decisions sampled per seed), days 19–25 test (2 M decisions). The player’s style vector is computed from the training days only (`bc.py`).

Table 4: Action prediction on real decisions (3 seeds).

Model (test days 19–25)	accuracy	NLL (nats)	macro-F1
majority class (legal-masked)	0.7015	—	—
frequency table (street, facing, position, raises, players left)	0.7059	—	—
logistic regression	0.7081 ± 0.0001	0.826	0.336
MLP 256-128	0.7158 ± 0.0001	0.800	0.370
MLP + the player’s style vector	0.7197 ± 0.0002	0.751	0.407

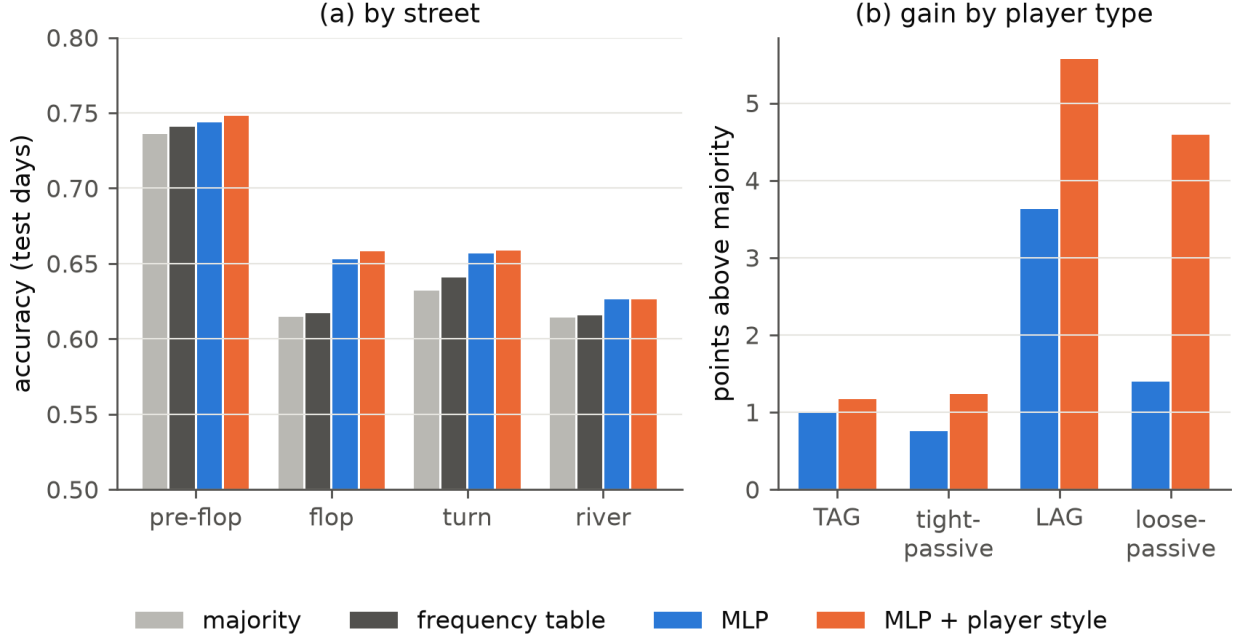


Figure 3: Behavioral cloning on the iPoker test days: (a) accuracy by street for the majority baseline, the frequency table, the MLP and the MLP with the player’s style vector (3-seed means); (b) accuracy above the majority baseline by player type. Knowing the player helps most for the loose players.

Results. The plan’s target is met and says nothing: 55 % of test decisions are folds and the majority baseline alone scores 70.2 %. The MLP adds 1.4 points over the majority baseline and the player’s style adds 0.4 more, with a larger effect on calibration ($NLL -0.049 \pm 0.0002$). Pre-flop is the most predictable street (0.748 vs 0.626–0.659), as predicted, but the baseline has the same order. The button is the *least* predictable position (0.639) and early position the most (0.813). Raw accuracy orders the types TAG 0.786 > LAG 0.650 > loose-passive 0.642, as predicted; measured above the majority baseline the order reverses (TAG +1.2, loose-passive +4.6, LAG +5.6 points), and the style vector adds most for the loose players (+3.2 points for loose-passive, +1.9 for loose-aggressive; seed means in `derived.json`). On the Pluribus table, where every hole card is known, the same MLP reaches 0.818 and 0.845 with the style vector (majority 0.697; pre-flop 0.926).

Conclusion. Behavioral cloning on public state is a baseline for *what people do*, not a model of *why*: most of the signal is hidden in the cards. The useful numbers are the gains over the baselines and the calibration gain from knowing the player — the quantitative form of the C1 claim that a player representation improves prediction. Kumar et al. (2022) predict that offline RL would beat cloning on noisy, sparse-reward data like this; cloning is used here only as a description of behavior.

1.6 Experiment 4 — A player2vec-style embedding and re-identification

What we test. Does a self-supervised embedding of the action stream capture the individual player better than the hand-crafted statistics? (Raw step, Day 3.)

How. Each hand a player sits in becomes tokens — position $\times$ table size, then street $\times$ facing $\times$ multiway $\times$ action, then how the hand ended (180 tokens). A 4-layer Transformer encoder ($d = 128$) is trained on 256-token windows of each player’s stream with BERT-style masking, on days 1–18 only (110,569 windows, 15 epochs, 3 seeds). A player’s vector is the mean of the encoder outputs. The test needs no labels: embed each player from N hands of days 1–18 and from N other hands of days 19–25, and rank all players by similarity (`p2v.py`). The statistics vector is scored the same way.

Table 5: Recognizing the same player on unseen days (3 seeds).

N hands per side	players	chance (top-1)	embedding	statistics top-1
			top-1 / top-10	/ top-10
50	2,575	0.04 %	1.3 % / 8.5 %	0.3 % / 2.8 %
100	1,859	0.05 %	4.2 % / 17.9 %	1.6 % / 7.4 %
200	1,278	0.08 %	9.0 % / 31.0 %	4.1 % / 14.8 %
400	834	0.12 %	15.6 $\pm$ 0.2 % / 45.2 %	7.1 $\pm$ 0.2 % / 23.8 %

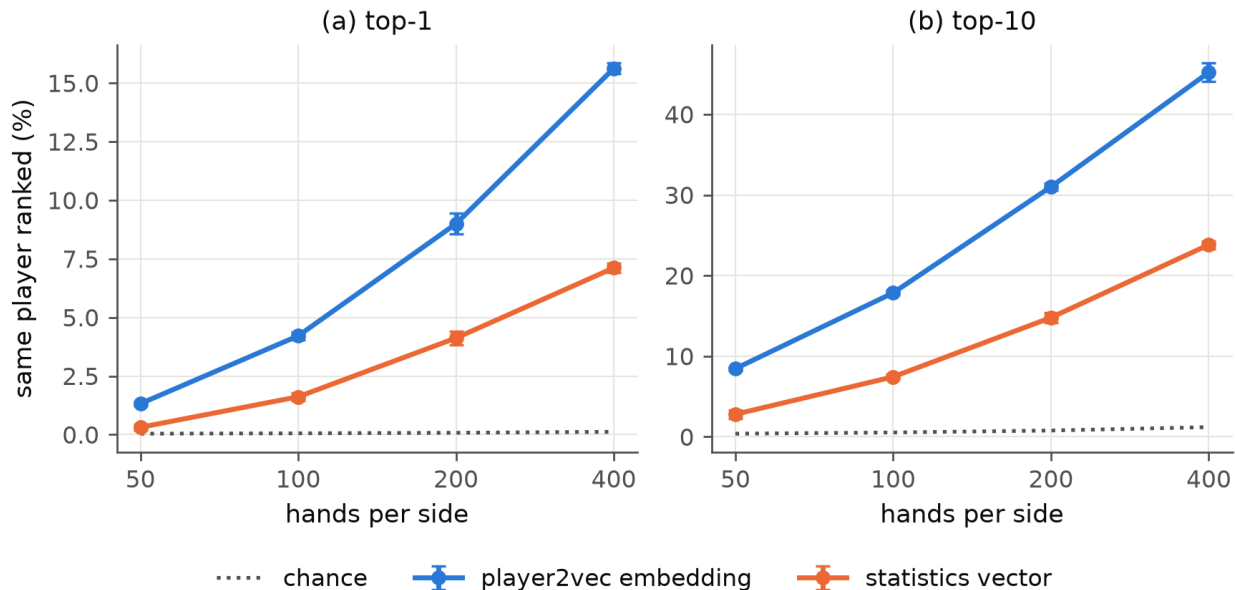


Figure 4: Re-identification of the same player on unseen days: share of players whose own later hands rank first (a) or in the top ten (b) among all candidates, for the player2vec embedding and the statistics vector, against the number of hands on each side (3 seeds, ± 1 SE). Chance is the dotted line.

Results. The embedding recognizes a player at least twice as often as the statistics at every sample size, and 130 times chance at 400 hands. Mean pooling beats the max pooling of the original paper (4.5 % top-1 at $N = 400$). A first run with 4 training epochs ended in different places for the three seeds (loss 2.62, 2.01, 2.63); with 15 epochs every seed passes the same drop to about 0.57–0.61, when the model learns the stream’s deterministic structure (positions rotate; the end token reveals a fold). The embedding also carries the classic axes: a 15-nearest-neighbor classifier recovers the VPIP $\times$ PFR quadrant with 0.90 accuracy, against 0.88 for the statistics vector that contains VPIP and PFR; cosine similarity is 0.86–0.89 within a quadrant and 0.74–0.76 between quadrants.

Conclusion. A token model of the raw stream holds more player-specific information than twelve tracker statistics. Re-identification is also the multi-accounting question — can one person be recognized across accounts? — and the answer here is “sometimes, from a few hundred hands”: useful for ranking candidates for review, far from proof.

1.7 Experiment 5 — Style clusters, temporal stability, online typing

What we test. Do the regulars form the four textbook archetypes; do players stay in their cluster over time (target > 70 %); and how fast does an online Bayesian model identify a player’s type?

(Raw step, Day 4; Math Flag on posterior concentration.)

How. k-means on 11 standardized statistics of the 2,906 regulars; silhouette for $k = 2-8$. Stability: each regular’s hands split into the first and second chronological half, both assigned to the full-data centroids; compared with a random split of the same hands (sampling noise alone) and with chance. Online typing: the four clusters become the types of a Bayesian model with per-hand Bernoulli events (VIP, PFR, limp, and 3-bet, c-bet, showdown and steal per opportunity); the posterior is updated hand by hand over a player’s first 500 hands, and the truth is the cluster of that player’s later hands (1,702 players with at least 1,000 hands; `clustering.py`).

Results. There is no natural four-cluster structure: silhouette is 0.16 at $k = 2$ and 3 and 0.13 at $k = 4$. k-means with $k = 4$ is stable across seeds (ARI 0.94, 0.97), but its clusters are two tight-aggressive groups, one loose-aggressive and one loose-passive group, with no tight-passive cluster (ARI 0.28 with the quadrants). Stability between the two halves is $73.6 \pm 0.2 \%$, above the plan’s 70 %, but a random split of the same hands gives $78.2 \pm 0.5 \%$: most of the switching is sampling noise at cluster borders, and genuine change over the month accounts for about five points. The embedding’s clusters are more stable (90.7 %), though its first half overlaps the training days. Online typing is correct for 49 % of players after 10 hands, 62 % after 100 and 68 % after 500, against 44 % for always guessing the largest cluster; 66 % of players reach a posterior of at least 0.9 on their later type that stays there within 500 hands, after a median 85 hands.

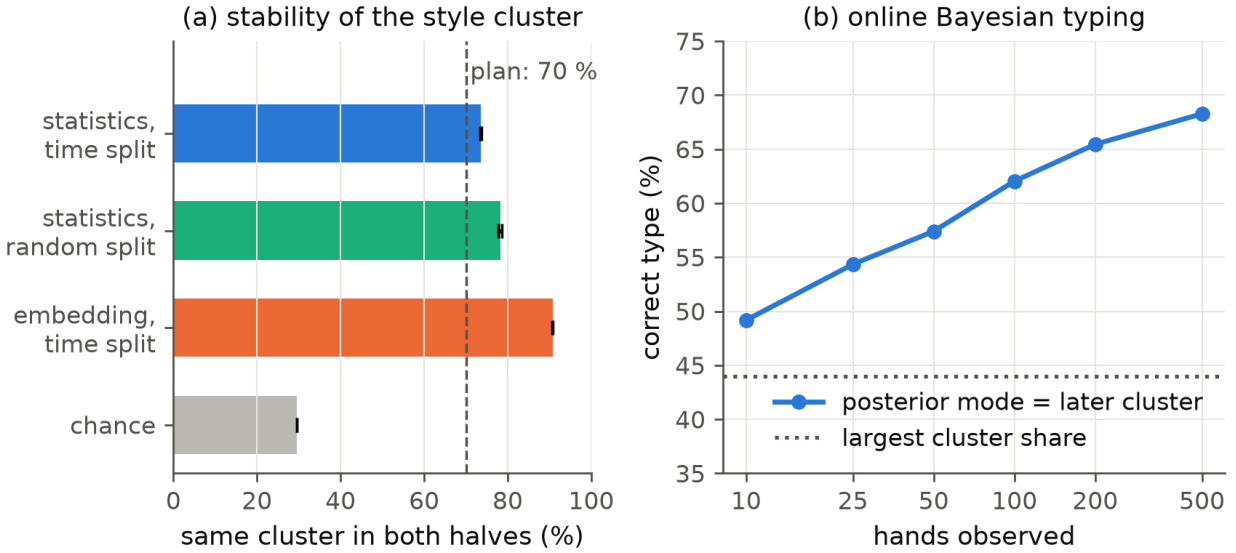


Figure 5: (a) Share of regulars in the same style cluster in both halves of their hands: chronological halves (statistics and embedding) and random halves (statistics), with chance; (b) online Bayesian typing: share of players whose posterior mode after n hands equals the cluster of their later hands, with the largest-cluster share for comparison.

Conclusion. Styles form a continuum with a dense tight-aggressive core; four archetypes are a convention, not a finding. Clusters still work as the prior of an online model: the “data → clusters → prior → Bayesian update” loop of Chapter 7 runs on real players, and it reaches a confident type within a hundred or so hands for most of them.

1.8 Part III — FAIR PLAY

1.9 Experiment 6 — Collusion injected into real sessions

What we test. Can a pairwise detector find colluding pairs, at what false-positive rate, and how much collusion does it need? (Raw step, Day 5: “detects 100 % of injected synthetic collusion; false positive rate on known-clean data < 5 %”.)

How. No labelled collusion exists — the published poker detectors were all validated on synthetic colluders (Mazrooei, Archibald & Bowling 2013; Bonjour, Aggarwal & Bhargava 2022). Here the colluders are injected into real sessions. Per seed, 40 pairs of real players who already share at least 300 hands and 30 heads-up confrontations are chosen, so co-occurrence cannot give them away. In a share q of the hands where the two end up heads-up, the rest of the hand is rewritten: *soft play* (both check or call to the end) or *chip dumping* (A bets the pot, B raises to three times, A folds). Every rewritten hand was accepted by the parser and by pokerkit (0 rejections; up to 3,306 hands per configuration). The detector compares each player with themselves: $\mathbf{z}_{\text{soft}}$ is the aggression deficit against the partner relative to the player’s heads-up aggression against everyone else, $\mathbf{z}_{\text{dump}}$ the chip flow between the two in their heads-up hands over its root-sum-square, and the *union* score is the larger of the two. Chapter 11’s help/harm score is carried over as a baseline with raw event counts (help = fold to the partner, harm = bet or raise at the partner). Negatives are the other real pairs of the same stratum, presumed clean (`collusion.py`).

The first run used all tables, and its top-ranked real pairs were all pairs from heads-up *tables*, with soft-play scores up to 142: the baseline compared play at a heads-up table with being left heads-up at a six-max table, two different games. The headline run therefore uses multiway tables only (≥ 3 seats); the null’s 99.9th percentile of the union score fell from 5.88 to 4.12.

Table 6: Union score, multiway tables, 2,802 real pairs in the stratum, 40 injected pairs $\times$ 3 seeds.

Type	q	AUC	recall at FPR 1 %	recall at FPR 0.1 %	FPR at 90 % recall
soft play	0.1	0.793 ± 0.013	$26 \pm 4 \%$	3 %	68 %
soft play	0.25	0.940 ± 0.011	$65 \pm 4 \%$	30 %	15 %
soft play	0.5	0.999	100 %	89 %	0.1 %

Type	q	AUC	recall at FPR	recall at FPR	FPR at 90 %
			1 %	0.1 %	recall
soft play	1.0	1.000	100 %	100 %	0 %
chip	0.1	0.635 ± 0.036	3 ± 1 %	0 %	83 %
dumping					
chip	0.25	0.853 ± 0.020	16 ± 5 %	1 %	34 %
dumping					
chip	0.5	0.972 ± 0.007	62 ± 3 %	8 %	6.6 %
dumping					
chip	1.0	0.998	97 ± 2 %	50 %	0.4 %
dumping					

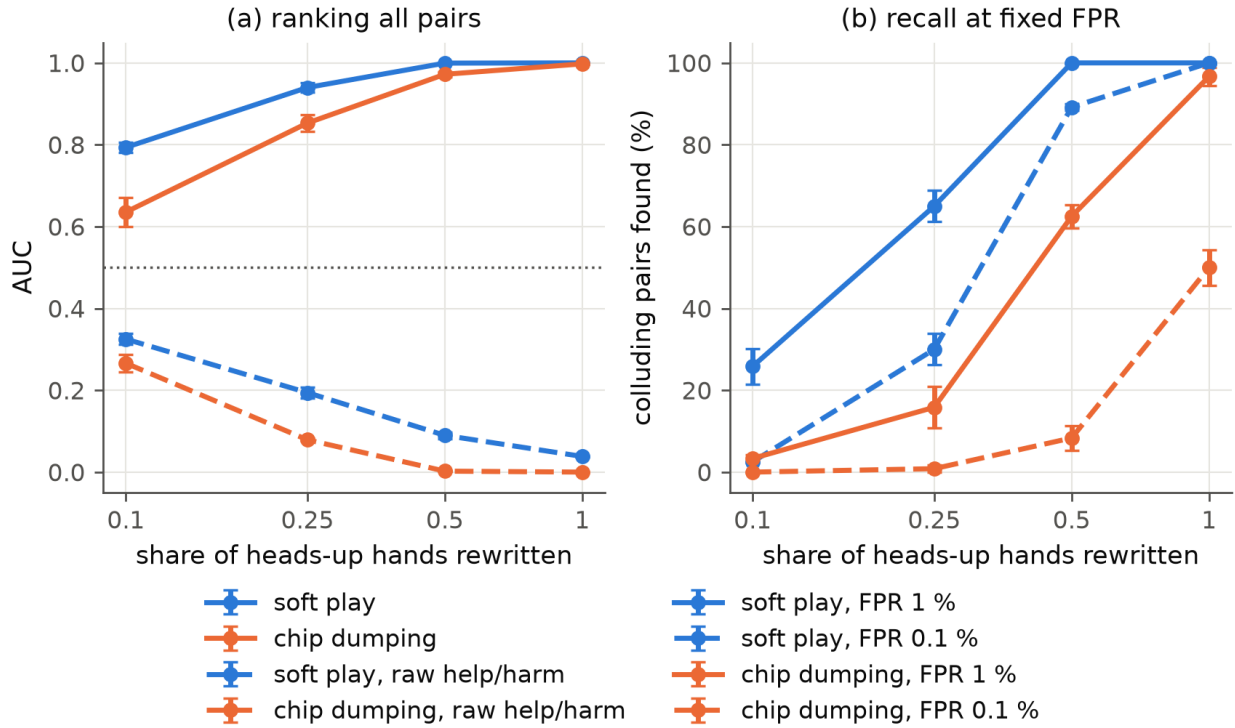


Figure 6: Collusion injected into real iPoker sessions (multiway tables, 3 seeds, ± 1 SE): (a) AUC of the union detector and of Chapter 11’s raw help/harm score against the share of heads-up hands rewritten; (b) share of colluding pairs found at a false-positive rate of 1 % and 0.1 % among 2,802 real pairs.

Results. Soft play is easier to find than dumping at every intensity. The plan’s target is met for soft play once half the confrontations are soft (100 % recall at 1 % FPR; FPR 0.1 % at 90 % recall) and for dumping only when every confrontation is a dump (FPR 0.4 % at 90 % recall). Light collusion

($q = 0.1$) is mostly invisible. Detection grows with the sample: at the null’s 99th percentile, soft play at $q = 0.25$ is found for 48 % of pairs with at most 39 heads-up confrontations and 83 % of pairs with more. Chapter 11’s help/harm score, with raw counts, is inverted (AUC 0.04–0.33): soft play removes the partner’s folds as well as the bets, and dumping adds bets. The all-tables run gives the same picture with weaker dumping detection (AUC 0.548 to 0.988). The highest real multiway scores (7.29, 4.71, 4.16) exceed the null’s 99.9th percentile; they are candidates for review, not findings.

Conclusion. The detector works only after two corrections — compare each player with their own behavior, and compare like with like (table format). Both are exactly the corrections a deployment on Playtech data would need, and the injection harness is the tool to calibrate them there.

1.10 Experiment 7 — Can behavior alone pick out Pluribus?

What we test. Whether the pipeline, without labels, flags the one bot in the Pluribus games. (Phase 1 goal “whether a bot is playing”; Math Flag “bots have unusually low loss”.)

How. Each player’s hands are cut into blocks of 250 (235 blocks, 40 of them Pluribus) and 500. Five unsupervised detectors are fitted on all blocks, without the label: Isolation Forest on HUD statistics, on HUD statistics plus the features the Pluribus paper points at (donk bets, limps outside the small blind, bet-size variety), and on the embedding; low decision-token loss under the iPoker-trained embedding model; and Mahalanobis distance from 250-hand blocks of iPoker regulars (`botdetect.py`, 3 seeds).

Results. None of them finds Pluribus. AUC (Pluribus blocks against human blocks, $B = 250$): HUD 0.415 ± 0.002 , HUD + tells 0.463 ± 0.007 , embedding 0.298 ± 0.011 , low loss 0.475 ± 0.007 , distance to iPoker 0.347; Pluribus ranks 10th to 14th of 14 by mean block score. Every AUC is below 0.5: Pluribus is *more* typical of this table than the average professional. Its decision-token loss, 0.691, sits inside the human range (0.529–0.829). Looking with the label, two features do separate it. Pluribus bets from a small menu of sizes: among 50 of its post-flop bets, a share of 0.141 are distinct sizes, against 0.324–0.670 for the professionals and a median 0.458 for iPoker regulars (16 of 1,275 regulars are at or below Pluribus). And it randomizes given its cards: the conditional entropy of its pre-flop action given position, situation and exact hand is 0.143 nats, the highest of the 9 players with enough data (humans 0.059–0.107). Its donk-bet rate (2.1 % against 0.9 %) matches the paper’s remark and is too small to detect with.

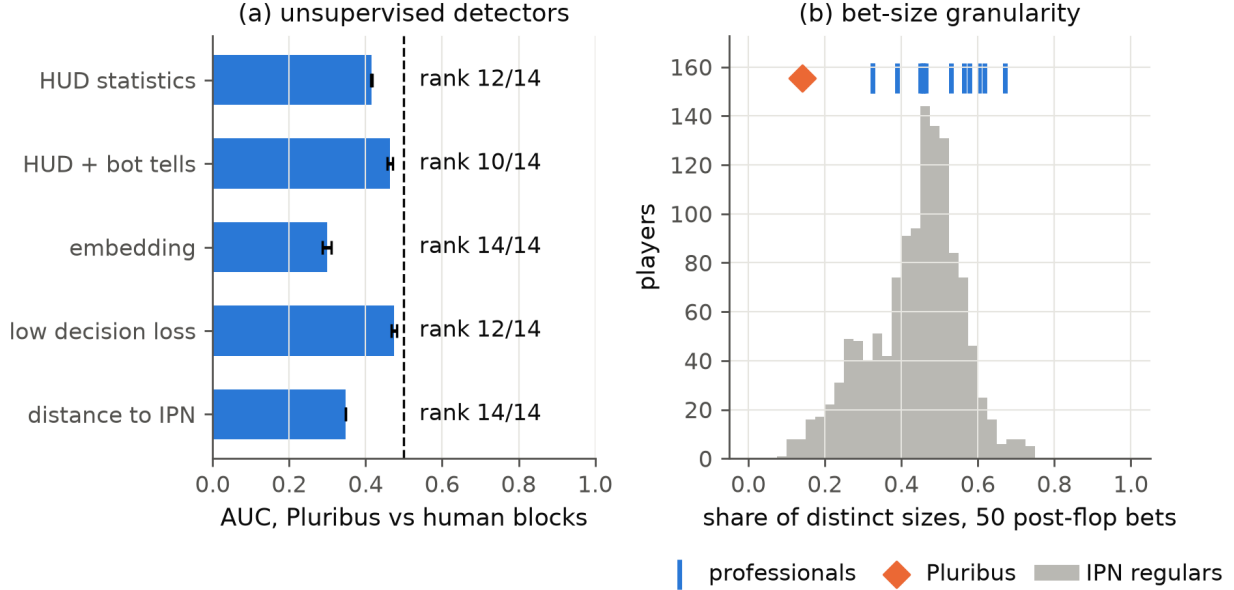


Figure 7: (a) Five unsupervised detectors on 250-hand blocks of the Pluribus games: AUC of Pluribus’s blocks against the professionals’ (3 seeds) and Pluribus’s rank among 14 players by mean score; (b) share of distinct bet sizes among 50 random post-flop bets of each player: iPoker regulars (histogram), the 13 professionals (ticks) and Pluribus.

Conclusion. A strong equilibrium-style bot is not an outlier in behavior space, and a detector built on “unusual” or “unusually consistent” will miss it. The two signals that work — a discrete bet-size menu and randomization conditioned on private cards — were found with the label on one bot. They are hypotheses to test on the Playtech data, not validated detectors.

1.11 Validation summary

Table 7: Raw-step validation targets against the measurements.

Raw-step target	Measured	Verdict
Parse 100 % of well-formed hands	99.77 %; every rejected hand is incomplete	met
Validator catches injected errors	own 100 % on 10 types; pokerkit misses duplicates, truncation	met (own)

Raw-step target	Measured	Verdict
BC accuracy > 55 %	71.97 % against a 70.15 % majority baseline	met, uninformative
TAG > LAG > Fish predictability	raw yes; above baseline reversed	not as intended
Embedding matches VPIP × PFR; intra > inter	0.90 15-NN accuracy; cosine 0.87 vs 0.75	met
Four clusters = four archetypes	2 TAG-like, LAG, loose-passive; silhouette 0.13	not met
Temporal stability > 70 %	73.6 % (random split 78.2 %)	met, noise-limited
Collusion 100 % at < 5 % FPR	soft play from q = 0.5; dumping at q = 1	partly

1.12 Prediction ↔ reality reconciliation

“BC accuracy > 55 % (random = 14 %).” *Measured:* 72 %. *Reconciliation:* the random baseline is the wrong comparison when 55 % of decisions are folds. Suspecting a leak first, I checked that the style vector uses training days only and that the test sample is disjoint in time; the gain over the majority baseline (1.8 points) is the honest figure.

“Button most predictable; TAG > LAG > Fish.” *Measured:* the button is the least predictable position; raw accuracy follows the fold rate of each group, and above the baseline the loose players gain most. *Reconciliation:* the predictions confused “easy to predict” with “folds a lot”. The lesson survives in a sharper form: knowing the player matters most for players far from the population.

“Four k-means clusters are TAG, LAG, Nit, Fish.” *Measured:* silhouette ≤ 0.16 , two TAG-like clusters, no tight-passive one. *Reconciliation:* not a bug — the four archetypes are regions of a continuous map, and k-means splits the dense tight-aggressive core instead.

“Bots have unusually low loss.” *Measured:* Pluribus’s loss is mid-range. *Reconciliation:* a bot that plays a fixed mixed strategy is predictable in its *distribution*, not in its actions; low loss would flag a deterministic script, not an equilibrium-style bot.

The first collusion run flagged heads-up tables. *Reconciliation:* a baseline mismatch in the detector, not collusion; fixed by format-matched pair tables. The first run is kept (`collusion.json`).

player2vec seeds disagreed after 4 epochs. *Reconciliation:* undertraining; with 15 epochs all seeds converge and re-identification improves (top-1 at N = 400 from 10–13 % to 15.6 %).

1.13 Trustworthiness and sample adequacy

- **Pipeline.** Two independent replays agree on payoffs in every comparable hand except a known, explained class; injected errors are caught at 100 %; every injected collusion hand is legal under both engines.
 - **Seeds.** All learned results have three seeds with standard errors much smaller than the differences reported. The BC test set (2 M decisions) and the reliability samples are large.
 - **Small samples where it matters.** Bot detection has one bot; the collusion false-positive rate is measured against 2,802 real pairs *presumed* clean; the gap reference is Pluribus’s own frequencies over 10,000 hands (binomial SE about 0.01–0.02 per cell).
 - **Missing information.** Stacks, winnings and most cards are absent from the iPoker records; statistics that need them (win-at-showdown, EV) are not reported.
-

1.14 Limitations (ranked by how much they affect the conclusions)

1. **2009 iPoker is not Playtech today.** Player populations, stakes and software have changed; every threshold here must be re-measured on the Playtech data.
 2. **Synthetic collusion.** Two scripted patterns in heads-up confrontations; information sharing between colluders (the most common form) is not modelled and would not show in these signals.
 3. **One known bot.** The two features that separate Pluribus were found with its label.
 4. **No EV.** The gap to Pluribus is a frequency distance; turning it into an exploitation value needs a game model this chapter does not have.
 5. **No Decision Transformer on real data** (optional in the brief): there is no simulator of these tables to evaluate a return-conditioned policy, so only hindsight leakage could be measured.
-

1.15 Conclusions and research directions

Conclusions. A parser with two validators replays 99.77 % of two million real hands and exposes three data conventions that would otherwise have corrupted every statistic. Statistics need between 100 and more than 1,000 hands to become reliable. Knowing the player improves prediction, and a self-supervised embedding recognizes players on unseen days about twice as well as the statistics. Styles form a continuum; clusters still serve as a prior for online typing. A detector that compares each player with their own behavior finds strong collusion at a 1 % false-positive rate and misses light collusion. Generic anomaly detection does not find Pluribus.

Research directions.

- Re-run every experiment unchanged on the Playtech data, starting with the reliability curves and the collusion null distribution.
 - Add information-sharing collusion (colluders who see each other's cards) to the injection harness.
 - Test the two bot signals (bet-size menu, conditional randomization) as pre-registered detectors on data with more than one known bot.
 - Couple the online Bayesian typing to the safe-exploitation machinery of Chapter 8, and use the collusion signals inside an agent's decision (C2), not only as an offline report.
-

1.16 Reproduction

From `implementation/step13/implementation/` with the project `.venv` active (runtimes on the RTX 5090 machine):

```
python build_dataset.py --source IPN100          # 2-3 min
python build_dataset.py --source PLURIBUS
python validator.py --n 2000                     # ~6 min
python player_stats.py                           # ~4 min
python bc.py && python bc.py --source PLURIBUS --epochs 40 --bs 512
python p2v.py --epochs 15                       # ~16 min
python clustering.py                             # ~6 min
python botdetect.py                             # ~3 min
python collusion.py && python collusion.py --pairs mw # ~8 + 9 min
python derived.py && python plotting.py          # figures from the saved JSON only
```

The data is fetched with a sparse clone of github.com/uoftcprg/phh-dataset (command in `EXECUTION_NOTES.md`) to `D:/datasets/phh-dataset`; parsed arrays are cached in `D:/datasets/step13_cache`.